

АКОС 10

Сети 1

Модель OSI

Модель					
Уровень (layer)		Тип данных (PDU)	Функции	Примеры	Оборудование
Host layers	7. Прикладной (application)	Данные	Доступ к сетевым службам	HTTP, FTP, POP3, SMTP, WebSocket	Хосты (клиенты сети), Межсетевой экран
	6. Представления (presentation)		Представление и шифрование данных	ASCII, EBCDIC, SSL, gzip	
	5. Сеансовый (session)		Управление сеансом связи	RPC, PAP, L2TP, gRPC	
	4. Транспортный (transport)	Сегменты (segment) / Датаграммы (datagram)	Прямая связь между конечными пунктами и надёжность	TCP, UDP, SCTP, Порты	
Media layers	3. Сетевой (network)	Пакеты (packet)	Определение маршрута и логическая адресация	IPv4, IPv6, IPsec, AppleTalk, ICMP	Маршрутизатор, Сетевой шлюз, Межсетевой экран
	2. Канальный (data link)	Биты (bit)/ Кадры (frame)	Физическая адресация	PPP, IEEE 802.22, Ethernet, DSL, ARP, сетевая карта.	Сетевой мост, Коммутатор, точка доступа
	1. Физический (physical)	Биты (bit)	Работа со средой передачи, сигналами и двоичными данными	USB, RJ («витая пара», коаксиальный, оптоволоконный), радиоканал	Концентратор, Повторитель (сетевое оборудование)

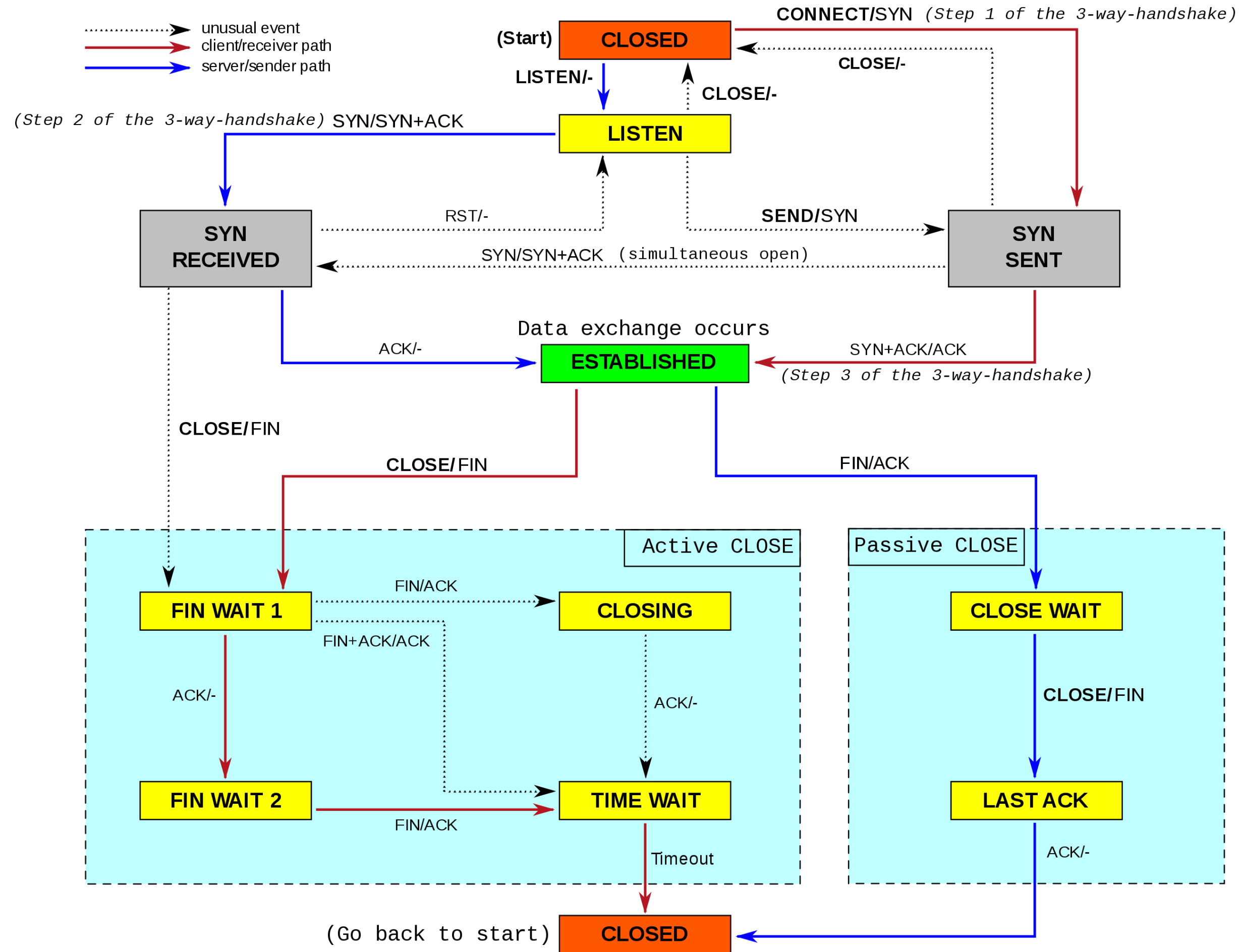
UDP

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Length																Checksum															
8	64	Data																															
12	96																																
⋮	⋮																																

TCP

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Sequence Number																															
8	64	Acknowledgement Number (meaningful when ACK bit set)																															
12	96	Data Offset				Reserved				C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window															
16	128	Checksum																Urgent Pointer (meaningful when URG bit set) ^[18]															
20	160	(Options) If present, Data Offset will be greater than 5. Padded with zeroes to a multiple of 32 bits, since Data Offset counts words of 4 octets.																															
⋮	⋮																																
56	448																																
60	480	Data																															
64	512																																
⋮	⋮																																

TCP



Socket

Socket is an endpoint for sending and receiving data across the network

socket

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(
    int domain,    // тип пространства имён
    int type,      // тип взаимодействия через сокет
    int protocol   // номер протокола или 0 для авто-выбора
)
```

Domain:

AF_UNIX - локальные UNIX-сокеты

AF_INET - IPv4 Internet protocols

AF_INET6 - IPv6 Internet protocols

Type:

SOCK_STREAM – bytestream (e.g. TCP)

SOCK_DGRAM – datagrams (e.g. UDP)

Protocol:

Обычно однозначно определяется по domain, можно выставить 0

connect

CLIENT

```
int connect(  
    int sockfd,                // файловый дескриптор сокета  
    const struct sockaddr *addr, // указатель на *абстрактную*  
                                // структуру адреса подключения  
    socklen_t addrlen          // размер реальной структуры sockaddr  
)
```

Attempts to make a connection to the socket that is bound to the address specified by addr.

sockaddr (AF_UNIX)

```
#include <sys/socket.h>
```

```
#include <sys/un.h>
```

```
struct sockaddr_un {  
    sa_family_t  sun_family;  // нужно записать AF_UNIX  
    char         sun_path[108]; // путь к файлу сокета  
};
```

sockaddr (AF_INET)

```
#include <netinet/ip.h>
```

```
struct sockaddr_in {  
    sa_family_t    sin_family; // address family: AF_INET  
    in_port_t      sin_port;   // port in network byte order  
    struct in_addr sin_addr;    // internet address (структура из одного поля uint32_t)  
};
```

•

Endian

```
#include <arpa/inet.h>
```

uint32_t htonl(uint32_t hostlong) - 32-битное из системного в сетевой порядок байт;

uint32_t ntohl(uint32_t netlong) - 32-битное из сетевого в системный порядок байт;

uint16_t htons(uint16_t hostshort) - 16-битное из системного в сетевой порядок байт;

uint16_t ntohs(uint16_t netshort) - 16-битное из сетевого в системный порядок байт.

int inet_aton(const char *cp, struct in_addr *inp) - конвертировать ip адрес

shutdown

```
#include <sys/socket.h>
```

```
int shutdown(int sockfd, int how);
```

- Закрывает соединение.
- close() недостаточен.
- how: SHUT_RD, SHUT_WR, SHUT_RDWR

bind

SERVER

```
#include <sys/types.h>  
#include <sys/socket.h>
```

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- Связывает сокет с конкретным адресом.
- По сигнатуре совпадает с connect.

listen

SERVER

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int listen(int sockfd, int backlog);
```

- marks the socket referred to by sockfd as a passive socket, that is, as a socket that will be used to accept incoming connection requests using accept
- socket должен иметь тип SOCK_STREAM или SOCK_SEQPACKET
- Backlog – длина очереди подключений

accept

SERVER

```
#include <sys/types.h>
```

```
#include<sys/socket.h>
```

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

- Тип сокета SOCK_STREAM, SOCK_SEQPACKET
- Взять одно входящее подключение из очереди или заснуть в ожидании соединения
- Вернуть sockfd для работы с подключением

send/recv

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

- Эквивалентны read/write, но с флагами

sendto/recvfrom

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_addr,  
socklen_t *addrlen);
```

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr  
*dest_addr, socklen_t addrlen);
```

- Для UDP
- Как send/recv, но ещё доп параметры о том, куда подключаться (нужны, так как ранее не было вызова connect)

getaddrinfo

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
#include <netdb.h>
```

```
int getaddrinfo(const char *node, const char *service,  
               const struct addrinfo *hints,  
               struct addrinfo **res);
```

Получить ip адрес по странице

Посмотреть порты

- \$ sudo netstat -tulpn
- (\$ sudo apt install net-tools)

```
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      4138041/nginx: mast
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      4138021/sshd: /usr/
tcp        0      0 127.0.0.1:41539         0.0.0.0:*               LISTEN      302063/node
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      4138066/systemd-res
tcp        0      0 0.0.0.0:12345           0.0.0.0:*               LISTEN      304680/./server
tcp6       0      0 :::80                  :::*                   LISTEN      4138041/nginx: mast
tcp6       0      0 :::22                  :::*                   LISTEN      4138021/sshd: /usr/
udp        0      0 127.0.0.53:53           0.0.0.0:*               4138066/systemd-res
udp        0      0 10.128.0.23:68          0.0.0.0:*               4138064/systemd-net
```

NetCat

- `nc 127.0.0.1 1234` – подключиться по TCP к серверу
- `nc -zv 127.0.0.1 2000-2010` – сканировать открытые порты
- `-l` – подключиться в режиме сервера

Python

[Документация](#)

Что почитать

[System Approach](#)